```python
import bpy
import math
import os
import tempfile

import numpy as np   # NumPy входит в поставку Blender


def header(n, title):
    print("\n" + "=" * 60)
    print(f"Кейс {n}. {title}")
    print("=" * 60)


# ---------------------------------------------------------------------
# Подготовка: очищаем сцену, чтобы скрипт можно было запускать повторно
# ---------------------------------------------------------------------
if bpy.context.object and bpy.context.object.mode != 'OBJECT':
    bpy.ops.object.mode_set(mode='OBJECT')
bpy.ops.object.select_all(action='SELECT')
bpy.ops.object.delete()
for block in (bpy.data.meshes, bpy.data.materials,
              bpy.data.cameras, bpy.data.lights):
    for item in list(block):
        if item.users == 0:
            block.remove(item)

scene = bpy.context.scene

# =====================================================================
#                ЧАСТЬ 1. Работа с Blender (Python API bpy)
# =====================================================================

# ---------------------------------------------------------------------
# Кейс 1. Создание примитива для VR-сцены
# ---------------------------------------------------------------------
header(1, "Создание примитива для VR-сцены")

bpy.ops.mesh.primitive_cube_add(size=2, location=(0, 0, 1))  # высота 1 м
cube = bpy.context.active_object
cube.name = "VR_Cube"

print("Создан объект:", cube.name)
print("Вершин:", len(cube.data.vertices))
```

```python
# ---------------------------------------------------------------------
# Кейс 2. PBR-материал для объекта (Principled BSDF)
# ---------------------------------------------------------------------
header(2, "PBR-материал (Principled BSDF)")

mat = bpy.data.materials.new(name="VR_Material")
mat.use_nodes = True
bsdf = mat.node_tree.nodes.get("Principled BSDF")
bsdf.inputs["Base Color"].default_value = (0.1, 0.3, 0.9, 1.0)  # синий (RGBA)
bsdf.inputs["Roughness"].default_value = 0.3

cube.data.materials.clear()
cube.data.materials.append(mat)

print("Материал назначен:", mat.name)
print("Материалов на объекте:", len(cube.data.materials))

# ---------------------------------------------------------------------
# Кейс 3. Настройка стереокамеры для VR (multiview)
# ---------------------------------------------------------------------
header(3, "Стереокамера для VR (multiview)")

bpy.ops.object.camera_add(location=(0, -8, 1.6),
                          rotation=(math.radians(90), 0, 0))
cam = bpy.context.active_object
cam.name = "VR_Camera"
scene.camera = cam

scene.render.use_multiview = True
scene.render.views_format = 'STEREO_3D'
cam.data.stereo.interocular_distance = 0.065   # 65 мм

print("Стерео включено:", scene.render.use_multiview)
print("Межзрачковое расстояние, м:",
      round(cam.data.stereo.interocular_distance, 3))

# ---------------------------------------------------------------------
# Кейс 4. Оптимизация меша под 90 FPS (модификатор Decimate)
# ---------------------------------------------------------------------
header(4, "Оптимизация меша (Decimate)")

bpy.ops.mesh.primitive_ico_sphere_add(subdivisions=3, radius=1,
                                      location=(4, 0, 1))
sphere = bpy.context.active_object
sphere.name = "VR_Sphere"
```

```
tris_before = len(sphere.data.polygons)          # 20 * 4^3 = 1280
ratio = 0.25
target = int(tris_before * ratio)

dec = sphere.modifiers.new(name="Decimate", type='DECIMATE')
dec.ratio = ratio
bpy.context.view_layer.objects.active = sphere
bpy.ops.object.modifier_apply(modifier=dec.name)

print("Было треугольников:", tris_before)
print(f"Целевое число (ratio {ratio}):", target)
print("Фактически после Decimate:", len(sphere.data.polygons))

# ----------------------------------------------------------------------
# Кейс 5. Инстансинг: расстановка объектов циклом
# ----------------------------------------------------------------------
header(5, "Сетка объектов 5x5")

rows, cols, step = 5, 5, 2.0
count = 0
for i in range(rows):
    for j in range(cols):
        bpy.ops.mesh.primitive_cube_add(size=0.5,
                                        location=(i * step + 10, j * step,
0.25))
        tile = bpy.context.active_object
        tile.name = f"Tile_{i}_{j}"
        count += 1

print("Создано объектов:", count)

# ----------------------------------------------------------------------
# Кейс 6. Освещение VR-сцены (источник Sun)
# ----------------------------------------------------------------------
header(6, "Источник света Sun")

bpy.ops.object.light_add(type='SUN', location=(0, 0, 10))
sun = bpy.context.active_object
sun.name = "VR_Sun"
sun.data.energy = 4.0
sun.rotation_euler = (math.radians(45), 0, math.radians(30))

print("Источник света:", sun.name)
print(f"Тип: {sun.data.type} | Мощность: {sun.data.energy}")
```

```python
# ---------------------------------------------------------------------
# Кейс 7. Анимация вращения (ключевые кадры)
# ---------------------------------------------------------------------
header(7, "Анимация вращения")

scene.frame_start = 1
scene.frame_end = 60

keys = 0
cube.rotation_euler = (0, 0, 0)
cube.keyframe_insert(data_path="rotation_euler", index=2, frame=1)
keys += 1
cube.rotation_euler[2] = 2 * math.pi            # полный оборот
cube.keyframe_insert(data_path="rotation_euler", index=2, frame=60)
keys += 1

print(f"Анимация: кадры {scene.frame_start} - {scene.frame_end}")
print("Ключевых кадров вставлено:", keys)

# ---------------------------------------------------------------------
# Кейс 8. Экспорт сцены в glTF (GLB)
# ---------------------------------------------------------------------
header(8, "Экспорт в glTF (GLB)")

out_path = os.path.join(tempfile.gettempdir(), "vr_scene.glb")
bpy.ops.export_scene.gltf(filepath=out_path, export_format='GLB')

print("Экспортировано в:", out_path)

# =====================================================================
#           ЧАСТЬ 2. Программирование для VR (расчёты на Python)
# =====================================================================

# ---------------------------------------------------------------------
# Кейс 9. Стереокамеры: позиции левого и правого глаза
# ---------------------------------------------------------------------
header(9, "Позиции камер левого и правого глаза")

head = np.array([0.0, -2.0, 1.6])     # позиция головы
right = np.array([1.0, 0.0, 0.0])     # локальная ось «вправо»
ipd = 0.065

left_eye = head - right * ipd / 2
right_eye = head + right * ipd / 2
dist = np.linalg.norm(right_eye - left_eye)
```

```python
print("Левый глаз: ", left_eye)
print("Правый глаз:", right_eye)
print(f"Расстояние между камерами: {dist:.3f} м")

# ----------------------------------------------------------------------
# Кейс 10. Поле зрения (FOV) и размер объекта в кадре
# ----------------------------------------------------------------------
header(10, "FOV и размер объекта в кадре")

fov = 100.0
distance = 5.0
obj_width = 2.0

view_width = 2 * distance * math.tan(math.radians(fov) / 2)
share = obj_width / view_width * 100

print(f"FOV = {fov} град, расстояние = {distance} м")
print(f"Ширина видимой области: {view_width:.2f} м")
print(f"Объект шириной {obj_width} м занимает {share:.1f}% ширины кадра")

# ----------------------------------------------------------------------
# Кейс 11. Кватернионы: поворот вектора взгляда
# ----------------------------------------------------------------------
header(11, "Кватернион: поворот вектора взгляда")

theta = math.radians(90)
axis = np.array([0.0, 0.0, 1.0])
w = math.cos(theta / 2)
x, y, z = axis * math.sin(theta / 2)
q = np.array([w, x, y, z])

# Матрица поворота из кватерниона
R = np.array([
    [1 - 2*(y*y + z*z), 2*(x*y - z*w),     2*(x*z + y*w)],
    [2*(x*y + z*w),     1 - 2*(x*x + z*z), 2*(y*z - x*w)],
    [2*(x*z - y*w),     2*(y*z + x*w),     1 - 2*(x*x + y*y)],
])

forward = np.array([0, 1, 0])
rotated = np.round(R @ forward, 3) + 0.0   # +0.0 убирает «-0.»

print("Кватернион (w,x,y,z):", np.round(q, 3))
print("Вектор взгляда до поворота:", [0, 1, 0])
print("После поворота на 90 по Z:", rotated)
```

```python
# ---------------------------------------------------------------------
# Кейс 12. Бюджет кадра для частоты VR-дисплея
# ---------------------------------------------------------------------
header(12, "Бюджет кадра")

for hz in (72, 90, 120):
    print(f"{hz} Гц -> бюджет кадра {1000 / hz:.2f} мс")

render_ms = 9.5
budget_90 = 1000 / 90
status = "успеваем" if render_ms <= budget_90 else "НЕ успеваем"
print(f"Рендер {render_ms} мс при 90 Гц ({budget_90:.2f} мс): {status}")

# ---------------------------------------------------------------------
# Кейс 13. Выбор уровня детализации (LOD) по расстоянию
# ---------------------------------------------------------------------
header(13, "Выбор LOD по расстоянию")


def select_lod(d):
    if d < 5:
        return "LOD0 (высокая детализация)"
    elif d < 20:
        return "LOD1 (средняя)"
    elif d < 50:
        return "LOD2 (низкая)"
    else:
        return "culled (не рендерится)"


for d in (2, 10, 35, 80):
    print(f"{d:>3} м -> {select_lod(d)}")

# ---------------------------------------------------------------------
# Кейс 14. Полигональный бюджет VR-сцены
# ---------------------------------------------------------------------
header(14, "Полигональный бюджет сцены")

budget = 1_000_000
per_obj = 8_000
max_objs = budget // per_obj
n_objs = 150
load = n_objs * per_obj
verdict = "в бюджете" if load <= budget else "ПРЕВЫШЕН бюджет"
```

```
print(f"Бюджет сцены: {budget:,} тр., на объект: {per_obj:,} тр.")
print("Максимум объектов в бюджете:", max_objs)
print(f"Сцена из {n_objs} объектов = {load:,} тр. -> {verdict}")

# ---------------------------------------------------------------------
# Кейс 15. Задержка motion-to-photon
# ---------------------------------------------------------------------
header(15, "Задержка motion-to-photon")

stages = {
    "Датчик (трекинг)": 2.0,
    "Обработка": 3.0,
    "Рендер": 11.0,
    "Вывод на дисплей": 4.0,
}
for name, ms in stages.items():
    print(f"{name}: {ms} мс")

total = sum(stages.values())
print(f"Суммарная задержка motion-to-photon: {total} мс")
print("Комфортно (< 20 мс)?", total < 20)

print("\nГотово: все 15 кейсов выполнены.")
```